Example answers - Data Platform Engineer / AWS DevOps

Interviewer reference, entirely in English. Prepared on 17 October 2026. Use alongside the 60-minute interview guide and the separate candidate exercise. Do not share this answer key with the candidate during the exercise.

How to use this document

These are examples of acceptable reasoning, not scripts the candidate must repeat. Personal-experience examples are hypothetical templates, not verified statements about this candidate. Replace bracketed details with evidence from their answers. Do not expect them to claim ownership of an entire platform to demonstrate competence in the part they actually delivered.

- **Regular/mid baseline:** sufficient understanding for a typical task in this role.
- **Stronger answer:** useful additional depth, not a compulsory checklist.
- **Probe further:** an incomplete or ambiguous answer that needs clarification.
- **Concerning if confirmed:** a misconception that remains after clarification.

Ask the main question, then choose follow-ups. This document covers more ground than can fit into one hour. Do not turn every example into an additional question. Assess communication throughout the English-language interview; there is no separate language test. Do not penalise accent or isolated grammar mistakes.

Interview block	Expected baseline
Q1 - Ownership	Identifies personal work, a decision, and a verified result
Q2 - Platform and security	Explains a plausible data flow, access boundaries, and failure handling
Q3 - Terraform	Understands state, change review, and partial failures
Q4 - CI/CD	Explains a working deployment flow and temporary AWS access
Q5 - Python	Explains early return, shared defaults and Boolean filtering in two short reviews, then discusses real automation and checks
Q6 - Operations	Investigates with evidence, restores processing, and communicates impact
Reserve questions	Probe only when relevant; do not make every topic mandatory

Q1. Ownership of an AWS project

Main question

> Your CV mentions an end-to-end cross-account S3 migration. Could you sketch the data flow and identify the components you personally designed, implemented, or operated?

Example regular/mid answer - hypothetical template:

> The source data was in [source account], and the consumers needed access through [destination account or platform]. The account and network baseline already existed. My responsibility was [specific Terraform changes and Python automation], while [another team] owned [other components]. I implemented [a concrete change], tested it in [environment], and checked [migration report and a read performed as the intended consumer]. I also investigated [one actual failure] and documented the steps for rerunning failed work.

Enough at this level: a clear boundary around personal work and one concrete example of implementation and verification. Approximate scale is acceptable.

Stronger answer: explains why one approach was chosen over another, what changed after testing, and which observable result established success.

Follow-up answers

Question	Example of an acceptable answer
How large was the migration?	"Approximately [objects/data volume/accounts]. I would check the migration report for exact figures. The important constraint was [throughput, window, or permissions]."
What already existed?	"The team had account provisioning and a base module. I added [specific resources or workflow]. I did not design the entire architecture."
What decision did you make?	"I compared [two realistic options] against [requirement]. We chose [option] because [reason], accepting [trade-off]."
What was the hardest issue?	"I observed [symptom], tested [hypothesis], found [evidence], changed [cause], and repeated the failing operation successfully."
What technical work preceded this role?	"I built [actual SQL/Python/reporting automation], maintained it for [users], and tested changes through [process]. That work did/did not involve cloud infrastructure."

Probe further: "We used Terraform and Lambda." Ask: "Which file, resource, or workflow did you change, and how did you check it?"

Concerning if confirmed: cannot describe their own contribution or distinguish an implemented solution from a proposal after being asked for a specific example.

Q2. AWS data platform and security

Main platform scenario

> A supplier delivers CSV and JSON to S3. Data engineers process the files, analysts query prepared data, and new data must be available within 15 minutes. How would you provision and operate this across dev, test, and production?

Fixed profile for this question: use the clarification card in the interview guide: 1-50 MB uncompressed files, about 10 MB on average, 10,000/day, and a peak of 1,000 over five minutes within that daily total. CSV and newline-delimited JSON require independent schema/type/required-field checks and light transformations, without large joins or external API calls. Full traffic applies to production; dev/test use synthetic samples in separate accounts. Up to ten analysts query prepared datasets.

The 15-minute requirement covers valid new deliveries from completed S3 upload to authorised SQL availability, including queueing and publication at the stated peak; BI refresh is excluded. Invalid files must be flagged within 15 minutes, retained and rejected in full, with an alert. A stable supplier `delivery_id` identifies resends even under another name. Raw retention is 30 days, prepared retention 90 days; one Region, KMS at rest and TLS in transit are assumed. See Q2 assumptions and scenarios.

Example regular/mid answer after clarification:

> I would confirm the profile and keep original deliveries in raw storage. S3 events, a queue and a Lambda function are one option for this independent, light processing, but I would benchmark representative 50 MB files and the stated arrival peak before committing to capacity. Events should identify objects rather than carry the whole file. I would validate the full input before publishing results, reject invalid files with a recorded reason and alert, and use the stable delivery ID with a durable mechanism to prevent duplicate effects. I would publish a queryable prepared dataset, automate deployment across isolated accounts, use restricted roles and encryption, and measure upload-to-query availability. A successful job is not sufficient unless valid data is accessible within 15 minutes at peak load.

An appropriately batched Glue design is also acceptable if its complete path meets the same target and the candidate explains the operational and cost trade-off. Do not require Lambda, Spark, SQS or Athena as a memorised answer.

Short follow-up tied to the profile: "A thousand files arrive over five minutes. What would you measure to check your design still meets the target?" Look for queue age, processing capacity, errors and actual SQL availability, with bounded concurrency and a peak test rather than an unsupported guarantee. With the assumed average, input is about 100 GB/day; 500 GB/day is the all-50-MB case. These figures do not establish execution time, memory or query cost.

Enough at this level: a coherent flow, reasons for the service choices, deployment automation, and awareness of permissions and failures. Naming every service or designing a complete enterprise platform is not required.

Stronger answer: defines what "ready" means, separates processing from publication, discusses replay and schema compatibility, and provides a reusable job module for data engineers. Distinguishes a successfully started job from a successfully completed and validated dataset.

Glue can provide data integration and processing plus a Data Catalog; Athena is one possible query layer. This is an example architecture, not a required stack. AWS Glue overview.

Platform follow-up answers

Where would you keep source and prepared data?

> I would retain the original input so that processing can be repeated. Prepared data would have a separate location and publishing rules. I would define retention and access separately for each area. If source files can be replaced, I would agree how we identify the exact input version used by a run.

What triggers processing and handles dependencies?

> I would choose events or scheduling based on the arrival pattern and workload. A queue can buffer arrivals and separate ingestion from processing. If steps depend on each other, I need to track completion and failures rather than assume that submitting the next job means it finished successfully.

How would you choose Lambda versus Glue?

> Lambda could suit a bounded event-driven operation or orchestration step. Glue could suit larger data-processing work, particularly where the required engine and parallel processing fit the transformations. I would test runtime, memory, input size, throughput, and cost. I would not choose purely because one service is already familiar.

Do not require memorised service limits. An unfamiliar service can be a learning gap while the candidate still demonstrates sound selection criteria.

What about duplicate files or a schema change?

> I would define a stable identity for the input and make reprocessing safe. I would validate required fields and types. Invalid or incompatible data would be recorded and isolated, with an alert, rather than silently published. Compatible additions could be supported through a documented schema policy.

How would you backfill one day?

> I would select a bounded set of source data and record the run and code version. The output strategy must replace or merge the intended results safely instead of blindly appending them. I would validate the backfill before making it visible and check that concurrent normal processing cannot corrupt the result.

What reusable capability would you give data engineers?

> A versioned module or template accepting the job artifact, environment, approved inputs and outputs, and operational settings. It would provision roles, triggering, logs, and alarms consistently, with a documented deployment workflow and runbook. I would avoid making every job use an administrator role.

Cross-account access and KMS

> How would you restrict an analyst in another account to approved encrypted data, and investigate AccessDenied?

Example regular/mid answer:

> I would identify the exact role making the request and whether it accesses the data directly or assumes a role in the platform account. I would grant access to the approved location and the required operations only. I would check the identity permissions and S3 resource policy, then the KMS permissions for encrypted objects. I would reproduce the failed read as the consumer, capture its error and request context, and check applicable explicit denies rather than broadly adding permissions.

Stronger answer: distinguishes role trust from data access, direct cross-account access from assumed-role access, and relevant SCP, boundary, session, or endpoint restrictions. Considers Lake Formation only if the design uses it.

For direct use of a KMS key from another account, the key policy and an IAM policy in the caller's account must allow the operation. Assuming a role changes which principal must be evaluated. AWS cross-account KMS access.

Probe further: "I would check IAM." Ask: "Which identity, operation, resource, and policy would you inspect first?"

Concerning if confirmed: treats an Allow on one side as overriding any Deny, or proposes permanent broad access as the final solution.

Cost follow-up

> What could drive the largest costs, and how would you protect delivery time?

Example answer:

> I would measure costs by workload and environment before changing anything. In my design I would examine processing duration and capacity, unnecessary reruns, stored data and retention, query scan volume, and data movement. I would test a change against both cost per processed dataset and end-to-end freshness, using representative peak traffic. A cheaper configuration is not acceptable if it misses the agreed delivery target.

Expect relevant cost drivers and measurement, not current prices from memory.

Q3. Terraform in a team

Main question

> Walk through a platform change from PR to production. Where is state stored, and how do you prevent conflicting changes?

Example regular/mid answer:

> I would update a reviewed module and its environment configuration in Git, run validation and tests, then review the plan for the intended environment. State would use a controlled remote backend rather than a developer's laptop or Git. I would configure locking and restrict access to the state. Production deployment would require the agreed approval, and I would prevent conflicting applies for the same state. After deployment I would verify the resource configuration and a representative data flow.

Enough at this level: understands why state exists, protects it, reviews plans, separates environments, and recognises concurrent-change risks.

Stronger answer: discusses smaller state boundaries, reusable module versions, controlled provider versions, saved-plan provenance, and separate plan/apply permissions where appropriate. Understands that a plan can become stale.

The S3 backend has optional state locking using `use_lockfile`; it is not enabled just by choosing S3. Its older DynamoDB-based locking is deprecated. Accept an accurate description of an older setup. Terraform S3 backend.

Partial apply failure

> A bucket was created, but the policy failed. What next? Does reverting Git revert infrastructure?

Example regular/mid answer:

> I would inspect the error, the resources that actually exist, and the current Terraform state. Some changes may already have completed, so I would not assume everything was rolled back. I would fix the cause and review a new plan before applying again. Reverting the commit only changes the code; we still need to plan and execute the intended infrastructure change. I would protect any data already present.

Stronger answer: considers unusual state-write failures or orphaned resources and checks before importing or reconciling them. Does not assume every failed apply requires import or manual state editing.

Terraform apply.

Additional Terraform questions

Question	Example regular/mid answer	Additional depth
How would you separate environments?	"Separate configuration and state, appropriately scoped credentials, and clear deployment targets. A dev change must not implicitly modify production."	Separate accounts and smaller state scopes where justified; understands that naming or workspaces alone do not create a security boundary.
What about a manual console change?	"I would review a fresh plan to identify drift and agree whether to codify the change or restore the declared configuration. I would not overwrite an unexplained production change blindly."	Explains ownership, audit trail, and prevention of recurring drift.
What if the plan deletes a data-bearing resource?	"Stop and investigate why replacement or deletion is proposed, confirm the environment and resource identity, and assess data protection and recovery before approval."	Discusses provider changes, resource addressing, lifecycle protections, and a separately tested migration.
Does sensitive remove a secret from state?	"No. It can hide normal display output, but the value can still be in state. I would restrict and protect state and avoid exposing plan artifacts."	Knows that supported ephemeral/write-only mechanisms are distinct and must be checked for the resource and Terraform version.

Terraform sensitive-data handling.

Probe further: "I would rerun apply." Ask what they would inspect first. Rerunning after understanding and fixing the error can be correct.

Concerning if confirmed: deleting state as a routine fix, force-unlocking without checking whether another run is active, or assuming apply is an atomic transaction across all resources.

Q4. CI/CD, OIDC, and deployment tests

Main question

> How do you deploy Terraform and a versioned Lambda script or Glue job, test the change, and obtain AWS access from GitHub Actions?

Example regular/mid answer:

> A PR would run code checks and unit tests, validate Terraform, and produce a reviewable plan for the relevant environment. The workflow would use OIDC to obtain temporary AWS credentials for an appropriately scoped role. I would deploy a versioned job artifact with its infrastructure and configuration, test the end-to-end path in a non-production environment, and promote the approved version through the agreed production process. A failed test would stop promotion. I would keep enough deployment evidence to identify which code and configuration are running.

Enough at this level: an ordered, credible workflow with tests, approvals, temporary access, versioned artifacts, and post-deployment verification.

OIDC follow-ups

What prevents another repository from assuming the role?

> The AWS trust policy must restrict the trusted issuer and relevant token claims, including the audience and subject for the intended repository and deployment context. Production environment protection adds control over which changes can use that context. I would check the actual token format for this setup instead of copying a broad wildcard example.

Trust policy versus permission policy?

> The trust policy controls who can assume the role. The role's permission policies control what the resulting session can do. Successful assumption does not mean the session can read every bucket or use every KMS key.

What does `id-token: write` grant?

> It lets the GitHub Actions job request an OIDC token. AWS still decides whether the token may assume a role and what that role can access.

Stronger answer: explains PR trust boundaries, protects production from untrusted code execution, limits role scope, and distinguishes environment approval from token-claim validation. Exact claim strings are not required.

GitHub OIDC in AWS.

Testing and failed deployment follow-ups

Question	Example acceptable answer
Resources exist, but the consumer cannot read data. What test catches that?	"An integration read under the intended consumer role against representative encrypted test data, plus checks that forbidden locations remain inaccessible."
How would you check job/schema compatibility?	"Run the new artifact against representative fixtures and expected schemas, checking output fields, quality rules, and completeness before promotion."
Apply succeeded, but integration failed. What next?	"Stop promotion, establish the impact, and inspect the failure. Choose fix-forward or a controlled rollback based on what changed and whether reversing it is safe for the data."
What moves between environments?	"The reviewed artifact version or digest and the agreed infrastructure/configuration versions. Environment-specific values vary through controlled configuration."

Probe further: "We run pytest." Ask which identity is used, what behaviour is asserted, and whether the tests use mocks or actual AWS resources.

Concerning if confirmed: static administrator keys treated as the default CI solution without justification, or automatic production promotion after a failed end-to-end check.

Q5. Short Python review and automation experience

This block uses two short functions and verbal answers. The candidate does not need to implement a full solution, write tests, configure mocks, or know boto3. Allow four minutes for exercise 1, two for exercise 2, and four for real project experience. Exercise 1 has two bugs. The records are an illustrative fixture, not a raw AWS API response.

What does the current function return?

The first input record has status `SUCCEEDED`. The next two records have status `FAILED`. Assume a fresh function definition with no previous calls.

Example regular/mid answer:

> It returns an empty list. The return statement is inside the for loop, so the function exits after the first record. That record succeeded, so nothing has been appended. The two failed jobs are never checked.

What fixes the early return?

Example answer:

> Move return failed outside the loop. Then every job is examined before the function returns. This fixes only the first bug; the shared default accumulator still needs to be addressed.

What happens on successive calls after only that correction?

> After moving only return outside the loop, what do two successive calls to failed_jobs(jobs) return for the sample input? Assume a fresh function definition and no other calls. Explain the second bug and how to fix it.

This question is mandatory. Each call is required to return only its own results.

Example regular/mid answer:

> The default list is created once when the function is defined. Calls that omit failed reuse that same list, and append changes it. The first call collects the two failed names. The second adds those names again instead of starting fresh. I would remove the failed parameter and create a new empty list inside the function on every call.

Assuming only the return indentation has been fixed, these are the values printed immediately after each call in the stated order:

Call	Printed value
First `failed_jobs(jobs)`	`["load_orders", "load_products"]`
Second `failed_jobs(jobs)`	`["load_orders", "load_products", "load_orders", "load_products"]`
Then `failed_jobs([])`	The same four old names, despite empty input

The original early return masks accumulation with this sample because the first record succeeds. With an initial failed record, even the original function would append across calls. The default is reused when the argument is omitted; the list is not shared by every possible function or every explicit argument.

Additional depth: both calls return the same list object. If the first result is saved, it also shows four names after the second call. This is why the table describes values printed immediately, not independent snapshots stored earlier.

Complete correction

Interviewer reference:

```
def failed_jobs(jobs):
    failed = []
    for job in jobs:
        if job["status"] == "FAILED":
            failed.append(job["name"])
    return failed
```

The candidate can explain both corrections verbally. A `None` default with a fresh list created per call, a list comprehension, or another clear equivalent is acceptable if every call produces an independent result. No complete implementation or clever syntax is required.

What about edge cases?

For each row below, start from a fresh function definition. The corrected column assumes both bugs have been fixed.

Input	Original function	Corrected function
Sample: success, failure, failure	`[]`	`["load_orders", "load_products"]`
Empty list	`None`, because the loop never runs and the function reaches its end	`[]`
Successful jobs only	`[]` after examining just the first job	`[]` after examining every job
A failed job followed by another failed job	Only the first job's name	Both names

Example answer:

> After both fixes, an empty input and an input without failed jobs both return a new empty list, even after earlier failed jobs. In the original function, empty input returns None because there is no return outside the loop. I would also check multiple failures to make sure the function does not stop after the first result.

No written tests are required. Mentioning representative inputs and expected outputs is sufficient for this part. Missing fields are outside the stated input contract; do not turn the task into a validation framework.

Exercise 2 - jobs to retry

Use the second exercise in the candidate handout. Select only runs with status `FAILED` or `TIMED_OUT`. The current condition is `if job["status"] == "FAILED" or "TIMED_OUT":`.

What does the function return?

> It returns all four names: load_customers, load_orders, load_products, and load_prices. The intended result contains only load_orders and load_products, because only their statuses are FAILED or TIMED_OUT.

Why does the condition select successful and cancelled jobs?

> Python evaluates the comparison on the left of or. If that comparison is false, the non-empty string TIMED_OUT on the right is truthy, so the if body still runs. The right side is not another status comparison. I need to compare the status to both permitted values or check membership.

Interviewer reference - correction:

```
def retryable_jobs(jobs):
    retryable = []
    for job in jobs:
        if job["status"] in ("FAILED", "TIMED_OUT"):
            retryable.append(job["name"])
    return retryable
```

Also accept `job["status"] == "FAILED" or job["status"] == "TIMED_OUT"`, or another clear equivalent. No code entry is required.

Which inputs should be checked?

Input	Expected after correction
The four sample records	`["load_orders", "load_products"]`
One FAILED record, then one TIMED_OUT record	Both names, in input order
Only SUCCEEDED and CANCELLED records	`[]`
Empty list	`[]`

Baseline: predicts that all names are currently returned, explains the truthy string, proposes a correct condition, and names representative inputs and outputs. The candidate need not know operator implementation details or write tests.

Probe further: ask, "Does the string on the right compare itself to the status? What happens when the left comparison is false?" Record the help needed. Merely replacing `or` with `and` does not implement the required selection.

Additional depth: notes short-circuit evaluation and that `or` returns an operand rather than necessarily a Boolean. This is optional. Missing fields, retry execution, backoff, and AWS integration are outside the exercise.

Actual Python automation experience - 4 minutes

> Describe one Python automation you personally implemented. What did it do, how did you handle failures, and how did you check it before deployment?

Example regular/mid answer - hypothetical template:

> I wrote [an actual script or Lambda] to [perform a concrete operation]. Its inputs were [source/configuration], and it reported [result]. I handled [a specific expected failure] explicitly and made failures visible through [a job result, structured log, or alert]. Before deploying it, I checked [representative cases] and verified [a real integration] in a test environment. One bug I found was [actual example], and I confirmed the fix by [evidence].

These are prompts for evidence, not facts to assume about this candidate.

Follow-up: what about a partial failure?

> I would expose the successful and failed scope so the caller can decide whether to stop, retry, or publish the result. Printing an exception and reporting complete success would hide the problem from the operator.

Follow-up: how would you check code using external services?

> I would use controlled responses from a fake or mock to check my code's branching and failure handling, and a separate integration check for the real service configuration and permissions. For example, I would check normal input, empty input, and an API failure. I do not need to call AWS for every unit test.

Follow-up: is retrying a timeout always safe?

> No. The remote operation may have completed even if I did not receive its response. I would consider the operation's effect and how to identify or reconcile an earlier attempt before retrying it blindly.

Overall Python baseline: explains the early return, the shared mutable default and its effect across calls, and corrections that produce independent results in exercise 1. Explains and corrects the Boolean filter in exercise 2. Discusses concrete automation and checks from their own experience. Both exercises must be covered; record any prompts needed rather than treating shared defaults or the second exercise as a bonus. Implementation and test-writing are not required during this block.

Stronger answer: uses their own example to explain partial failures, safe retries, or meaningful unit and integration checks. Do not require building these mechanisms during the interview.

Probe further: if they name the indentation bug without explaining control flow, ask what the function returns for the first successful record. If they miss the default-list issue, ask where the list is created and whether the next call uses the same object. Naming the mutable-default rule without explaining repeated-call results is incomplete. If the project answer is just a list of libraries, ask for one actual operation and failure they handled.

Concerning if confirmed: cannot explain the loop/return behaviour after clarification or repeatedly treats hidden failures as successful automation. One small mistake alone is not enough to conclude that programming ability is weak.

These short exercises check code reading and basic reasoning. They do not establish the ability to implement production automation on its own. Record what was demonstrated in each snippet separately from the candidate's project account.

Q6. Operations, networking, and incident communication

First three checks

> CI is green and files arrive in S3, but the report has had no new data for 40 minutes. What are your first three checks?

Example regular/mid answer:

> First I would confirm the affected dataset, its last successful update, and whether the issue started with the deployment. Second, I would trace a known new input through event delivery and the queue to see where progress stops. Third, I would check the processing stage's invocations, logs, permissions, and output, then whether the report reads that output. For each hypothesis I want a specific observation. A green CI run or no Lambda errors does not establish that new data actually reached the consumer.

Enough at this level: a structured investigation with evidence and awareness of the consumer outcome. Exact console navigation or command spelling is not required.

Stronger answer: correlates object/run identifiers, compares behaviour before and after the deployment, limits further impact, and verifies recovery with freshness and reconciliation checks rather than just a healthy service status.

Duplicate processing

> The same message reaches Lambda twice. How do you prevent a duplicate effect?

Example regular/mid answer:

> I would assume retries are possible and use a stable operation identifier linked to the input or business event. The result should be written in a way that repeating the operation is safe, or completion should be tracked durably. I would not rely on a global Python set because another Lambda execution may run in a different process or after a restart.

Stronger answer: handles concurrent attempts and the failure window between writing the business effect and marking completion. A "processed" record written too early can lose work; a record written too late can allow duplication. Do not require a complete distributed protocol at the baseline level. AWS Lambda with SQS.

One failed message in a batch

Example answer:

> I would distinguish transient failures from invalid inputs and avoid retrying successful work unnecessarily. With the appropriate event-source configuration, partial batch responses can identify failed records for retry. Repeatedly failing inputs need a recorded path for investigation and controlled replay, such as a configured dead-letter queue. I would check visibility timeout, retry behaviour, and concurrency against processing time.

Stronger answer: notes that raising an unhandled error can fail the whole batch; if FIFO ordering matters, the handling strategy must preserve it. Exact response JSON is not required. AWS SQS event-source error handling.

Recovering the backlog and detecting incomplete data

Example answer:

> I would fix and verify the cause before replaying a bounded backlog. I would identify the affected inputs, control the replay rate, and avoid duplicating successful output. After replay, I would reconcile expected and processed inputs and check the consumer-visible dataset. I would monitor data freshness and completeness against the agreed target, with a named owner and a runbook. A successful job with zero useful output should not automatically satisfy the data-delivery requirement.

Stronger answer: distinguishes an idle source from a stuck pipeline, relates freshness to the supplier's expected delivery pattern, and separates technical job completion from business-data completeness.

Lambda in private subnets times out

Example regular/mid answer:

> I would confirm the target service and error, then check routes, DNS, security groups, and any relevant network ACL or endpoint configuration. For an internet destination I need an appropriate outbound path; for supported AWS services I can use the relevant VPC endpoint. I would not add IAM permissions merely because the connection times out. I would test connectivity and inspect any actual AccessDenied response separately.

Stronger answer: distinguishes an S3 gateway endpoint and its route-table association from a Secrets Manager interface endpoint, including endpoint security groups, private DNS, and endpoint policies. Understands that attaching Lambda to a public subnet does not automatically give it internet access. These details are additional depth, not a two-minute configuration exam. AWS Lambda VPC connectivity, Secrets Manager VPC endpoints, S3 gateway endpoints.

Stakeholder update

Example answer - hypothetical incident update:

> The reporting dataset has not received new data for 40 minutes, so users should treat the current report as out of date. New files are still arriving in S3. We are tracing where processing stops and checking the latest change. We have not yet confirmed the cause or whether all delayed inputs can be recovered automatically. Our next steps are to verify the failing stage, apply a controlled fix, and validate the backlog. I will provide another update in 15 minutes, even if the investigation is still ongoing.

The update interval is an example commitment, not a promised recovery time. Do not expect the candidate to invent a confident root cause or ETA.

Emergency change in a regulated environment

Example answer:

> I would explain the impact and use the agreed emergency-change process rather than silently bypass controls. I would involve the responsible approver, record the change and justification, perform the required validation, and confirm the outcome. Afterwards I would complete any agreed follow-up review and preserve the evidence. The exact steps depend on the organisation's approved procedure; I would not invent a universal GxP rule.

Baseline communication: understandable, structured, honest about uncertainty, and clear about next steps and ownership. Perfect grammar is not required.

Reserve questions

Use these as substitutions or deeper probes, not additional mandatory coverage.

R1. Moving a resource from CloudFormation to Terraform

Example regular/mid answer:

> I would inventory the resource and dependencies, write matching Terraform configuration, and plan a controlled ownership handover. I would ensure that removing it from CloudFormation does not delete the physical resource, then import it into the intended Terraform state and inspect the plan. Import is not enough by itself: the configuration must match the existing resource. The goal is one agreed management owner, with no unintended replacement.

Stronger answer: verifies retention settings and dependencies on a test case, avoids both tools actively changing the resource, and obtains a no-unintended-change plan before resuming normal deployments. Treat the sequence as resource-dependent, not a universal command recipe. Terraform import, CloudFormation DeletionPolicy.

R2. Infrastructure tests using administrator credentials

Example answer:

> A passing administrator test proves only that this identity could perform the operation. I would test the intended consumer role against an allowed dataset and verify that an unrelated location or forbidden operation is denied. I would also check that the failure is the expected authorization failure, rather than a wrong resource name or missing test fixture.

Baseline: recognises identity-specific behaviour. Bonus: covers encryption and cross-account access in representative fixtures without exposing real data.

R3. Comparing S3 logging architectures

Example answer - hypothetical template:

> I compared [actual options] against the questions users needed to answer, expected volume, retention, audit requirements, and acceptable query latency. I included ingestion, storage, and query or operational costs rather than looking only at a storage price. I validated [actual assumptions or sample]. My final output was [a recommendation or implemented change], and [owner] made the implementation decision.

Do not expect specific services, numerical savings, or a deployed outcome unless the candidate can substantiate them. Clearly separating an investigation from implementation is a positive signal.

R4. SQL reconciliation after migration

Example regular/mid answer:

> I would first agree the comparison scope: the same time window, filtering, business keys, and data freshness on both sides. I would compare counts, then identify missing keys and duplicate multiplicities. For matching keys I would compare relevant values with agreed handling of nulls and types. Matching row counts alone cannot prove equality because different missing or duplicated records may cancel out.

Stronger answer: uses grouped counts per key or dataset partition and appropriate joins, while accounting for late data and comparison semantics. Does not use DISTINCT to conceal unexpected duplicates.

R5. Docker troubleshooting

Example regular/mid answer:

> I would describe the actual problem the container solved in my project. For a failed container I would check its exit status and logs, image version, command, configuration, required secrets, resource limits, and access to dependencies. I would reproduce the issue with representative configuration and verify the fix rather than repeatedly restarting it.

Stronger answer: links the symptom to a specific runtime cause and discusses repeatable builds and least-privileged execution. Lack of Kubernetes depth is not an automatic failure when the actual role does not require it.

R6. S3 migration completeness and cutover

Example regular/mid answer:

> I would define the expected source scope and identify the version or cutoff being migrated. Historical objects need an explicit transfer or backfill approach, while ongoing changes need their own handling. I would reconcile expected and transferred objects, account for failures, and test reads as the destination consumer before cutover. If writes continue, I need an agreed way to catch up or establish a consistent handover point.

Follow-up answers:

- **Objects created before replication:** "I would not assume that enabling live replication automatically transfers them. I would choose a suitable historical transfer, such as Batch Replication where applicable, and verify the result."
- **Overwrites and versions:** "I would agree whether only the current version or historical versions are required and reconcile against that scope."
- **Counts versus content:** "Equal counts can hide different missing objects or incorrect contents. I need matching identities and an appropriate integrity check; an ETag is not universally the MD5 of the file."
- **Failed copies:** "Record a durable failure report, fix the cause, rerun the failed scope safely, and check the destination again."

AWS replication scope, AWS S3 Object and ETag.

For an encrypted-object AccessDenied follow-up, use the identity/S3/KMS reasoning in Q2. Do not require memorising the full replication-role policy.

R7. IAM access-key rotation

Example regular/mid answer:

> I would first justify why this consumer cannot use temporary credentials. For rotation I would create a new key while the old one still works, store it securely, update the consumers, and validate their actual operations. Then I would deactivate the old key, observe the result, and delete it only after the agreed checks. I would record progress so a retry does not blindly create another key. I would never put secret values in logs.

Failure after key creation but before secret storage:

> IAM cannot return that secret again. I would reconcile the keys and the recorded rotation state to identify an orphan from this attempt. Before cleaning it up or retrying, I would establish whether another attempt or consumer could be using it. The workflow needs a deliberate recovery path; blindly creating keys again may hit the user limit or disturb a valid rotation.

What did STS validation prove?

> GetCallerIdentity tells me which identity the credentials represent. I still need to test the intended operation, such as reading the approved encrypted object. Authentication and authorization are different checks.

What about a consumer caching the old secret?

> I would understand its refresh behaviour and validate its switch before retiring the old key. A grace period must be tied to consumer behaviour and monitoring, not just an arbitrary sleep. Deactivation gives a controlled checkpoint before permanent deletion.

Stronger answer: mentions the two-key limit including inactive keys, concurrent invocations, durable rotation identity, and recovery ownership. A complete automated rotation protocol is beyond the baseline expectation. AWS key rotation, AWS access-key limits and secret retrieval, AWS GetCallerIdentity.

R8. Exposed database password in GitHub - 2-4 minute reserve

Question:

> You discover that a database password was committed to a GitHub repository used by an AWS data pipeline. A teammate removes it in a new commit and says the problem is solved. Do you agree? What would you do next, and how would you check that the pipeline still works?

Adequate regular/mid answer:

> No. The password may still be in Git history or other copies. I would treat it as exposed and involve the incident and database owners so the credential is revoked or rotated promptly. The recovery needs to coordinate the real database credential, its stored secret and the consumers. I would verify that the old credential no longer works and that the intended job can perform an authorised operation with the replacement, without logging either value. History cleanup and checks for other copies would be coordinated separately. I would also address how the secret entered the repository.

The exact containment and rotation sequence depends on risk, the database and the organisation's process. Do not demand a zero-downtime procedure or two simultaneously valid passwords for every database. Updating a stored secret alone does not prove the database accepts the replacement. AWS: secret rotation.

Choose one follow-up:

Prompt	Expected reasoning
What if the repository is private?	Exposure is narrower, not automatically absent. Check who or what could read it; still invalidate the disclosed credential.
Secrets Manager was updated, but the job fails. What next?	Check the actual database credential, secret reference/version, cached connections, consumer configuration and exact error. Distinguish authentication, permissions and connectivity.
How would you reduce recurrence?	Keep secret values outside source code, restrict runtime access, avoid secret logging, and use appropriate scanning and review controls.

Probe further: "I would delete the file" or "I would force-push" without credential invalidation; "I would restart the job" without checking what it reads.

Stronger answer: identifies affected consumers and exposure window, coordinates containment, checks available audit evidence without claiming that missing logs prove no misuse, and verifies restored data delivery.

Concerning if confirmed: claims that a new commit or a private repository makes the original password safe, or proposes pasting it into chat/logs to debug.

No Git cleanup commands, implementation or real credentials are required. Use two minutes for the core answer, or up to four including one follow-up. Protect the candidate's final five minutes and retain the separately observed gap in any skipped skill. See the R8 interviewer guide in Polish. GitHub: sensitive-data removal.

Final calibration for the interviewer

An adequate regular/mid candidate should explain typical work independently, recognise common failure modes, and verify results. They can consult documentation for service-specific details and acknowledge gaps. They do not need to reproduce every stronger-answer point in this file.

Observation	How to interpret it
Correct basic answer with a real example	Evidence toward level 3 in that area
Correct direction but substantial prompts needed	Record the support required; often level 2 evidence
Detailed trade-offs and partial-failure handling	Additional evidence toward level 4, not a compulsory baseline
Unfamiliar API name but correct reasoning	Allow documentation; evaluate the underlying skill
Incorrect claim corrected after clarification	Record how much prompting was required and whether the corrected explanation is understood
Persistent basic misconception	Record the specific misconception and its operational consequence
Topic not covered	Mark N/O; do not infer inability

Use observed evidence, not similarity to these sentences. Keep technical skill, communication, and the formal experience requirement as distinct observations.